



ACTIVITY LAYOUTS



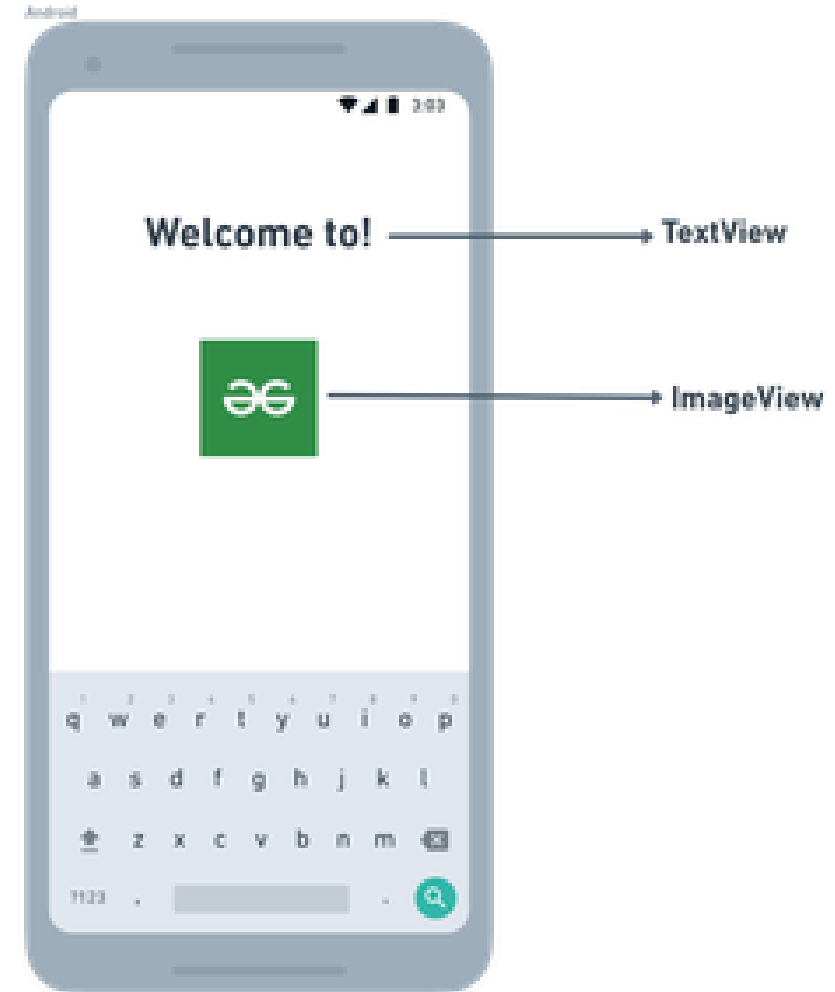
ANDROID LAYOUTS

- Android Layout is used to establish the user interface containing the **UI controls** and **widgets** that will be displayed on an Android application or activity screen.
- Every application is often a combination of **View** and **ViewGroup**.
- Hence, each activity comprises numerous user interface components: instances of **View** and **ViewGroup**.



VIEW

- A **View** is defined as the user interface which is used to create interactive UI components such as TextView, ImageView, EditText, RadioButton, etc., and is responsible for event handling and drawing. **They are Generally Called Widgets.**

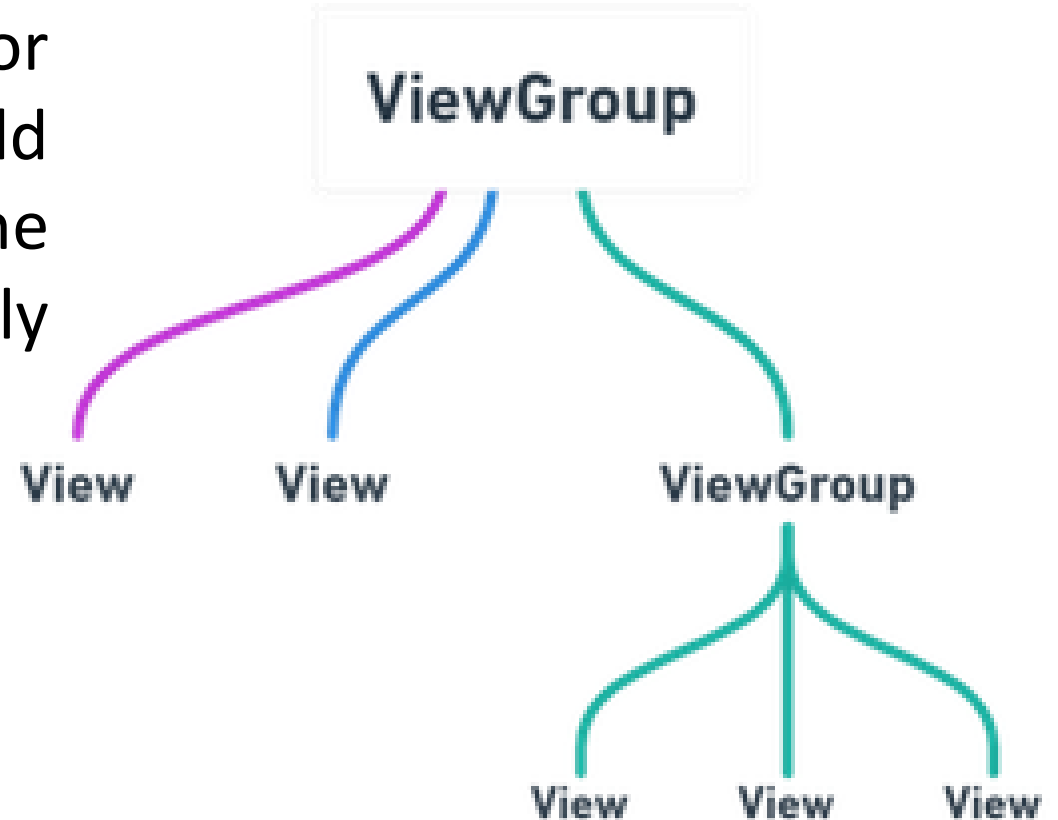




android

VIEW GROUPS

- A **ViewGroup** acts as a base class for layouts and layout parameters that hold other Views or ViewGroups and define the layout properties. They are Generally Called layouts.





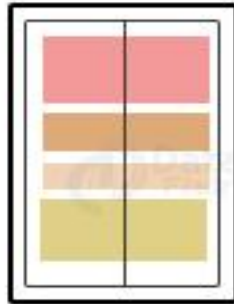
ANDROID ACTIVITY LAYOUTS

- The Android framework will allow us to use UI elements or widgets in two ways:
 - Use **UI elements in the XML file**
 - Create **elements in the Java/Kotlin file** dynamically

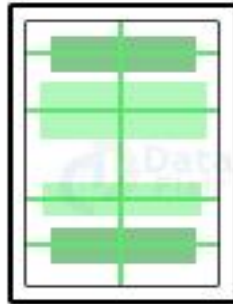


android

Types of Android Layouts



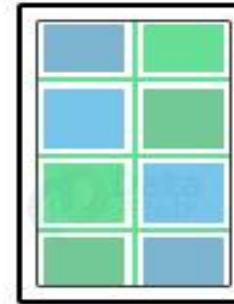
StackLayout



AbsoluteLayout



RelativeLayout



GridLayout



ContentView



ScrollView



Frame



TYPES OF UI (USER INTERFACE) LAYOUTS

■ 1. Linear Layout

- This view group aligns all of its children (Views or ViewGroups) in a single direction, **either vertically** or **horizontally**.
- The **android:orientation** attribute specifies the layout's direction.

```
<LinearLayout  
xmlns:android="http://schemas.android.com/apk/res/android"  
android:layout_width="match_parent"  
android:layout_height="match_parent"  
android:orientation="horizontal">  
<!-- Include other widget or layout tags here. These are  
considered "child views" or "children" of the linear layout -->  
</LinearLayout>
```

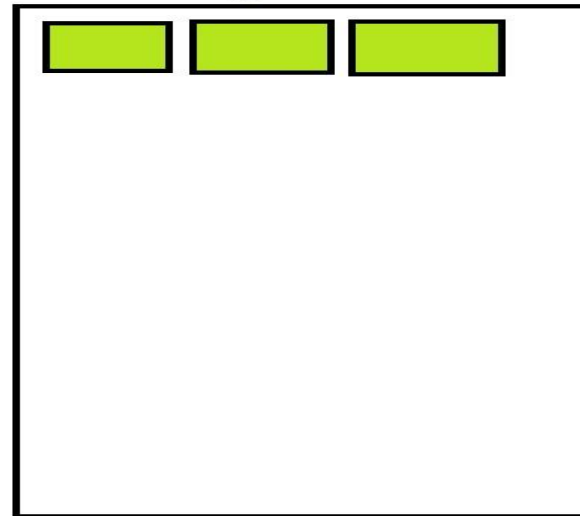
TYPES OF UI (USER INTERFACE) LAYOUTS

■ 1. Linear Layout

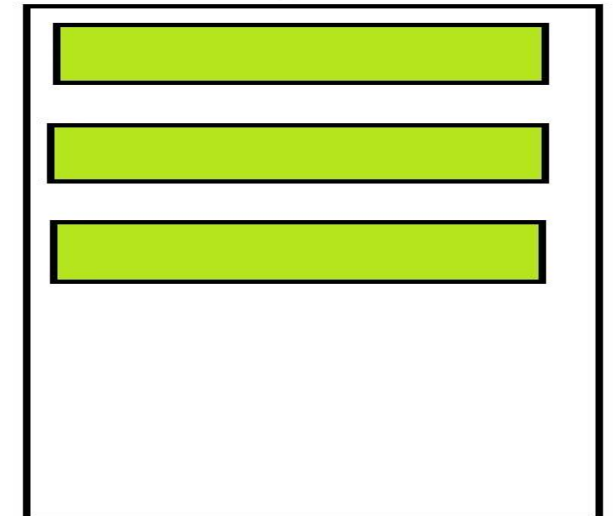
- Setting **android:orientation** helps specify if the child views are displayed in row or column format.



Linear Layout Horizontal



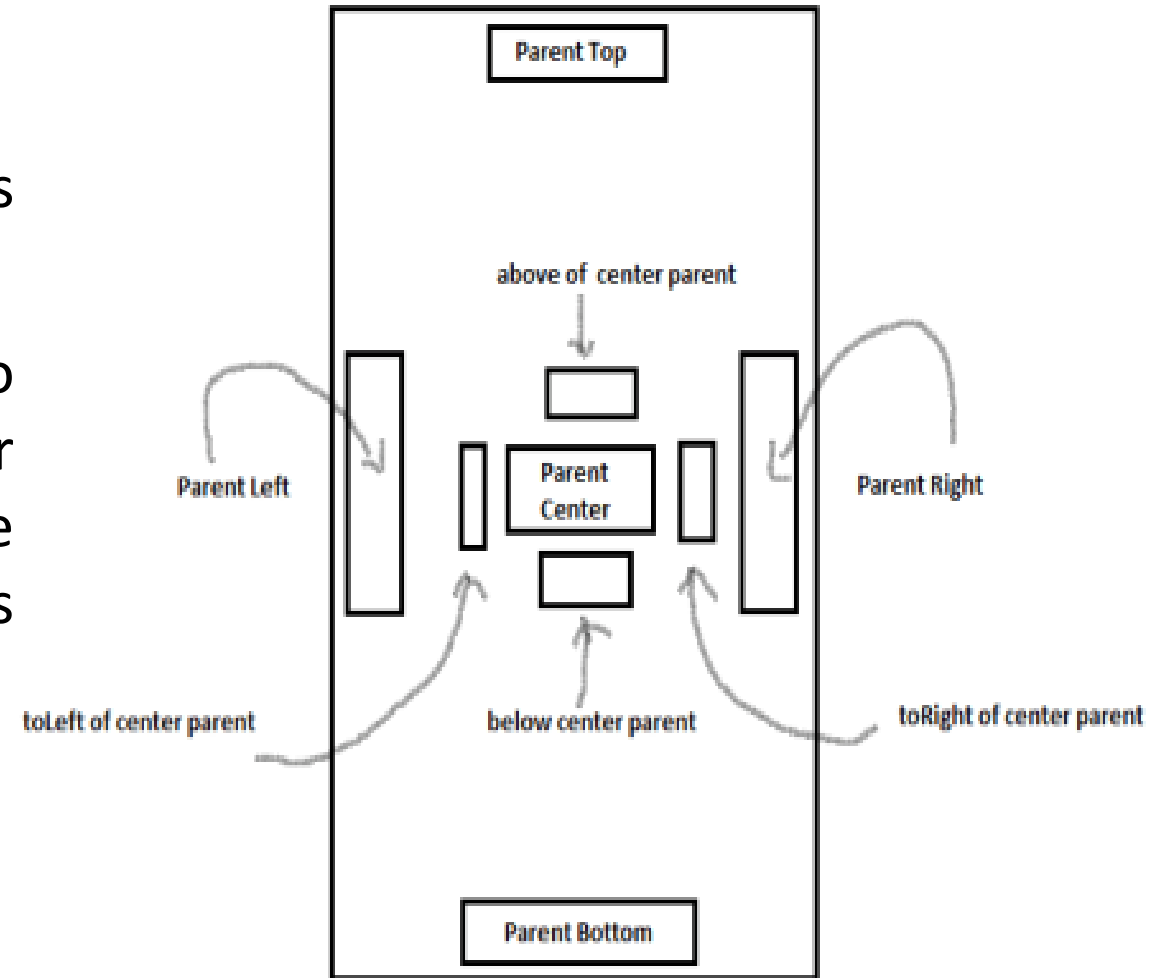
Linear Layout Vertical



TYPES OF UI (USER INTERFACE) LAYOUTS

■ 2. Relative Layout

- RelativeLayout is a view group that places its child views relative to one another.
- Each view's location can be set relative to sibling components (such as to the left or below another view) or relative to the RelativeLayout area of its parent (such as aligned to the bottom, left or center).



TYPES OF UI (USER INTERFACE) LAYOUTS

■ 3. Absolute Layout

- allows us to specify the exact location of the child views and widgets



Absolute Layout Works on X and Y Coordinate

TYPES OF UI (USER INTERFACE) LAYOUTS

■ 4. Frame Layout

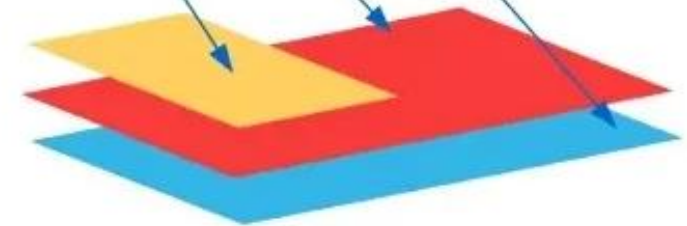
- FrameLayout is designed to display a single object in a section of the screen.
- In general, FrameLayout should only be used to host a single child view, as it can be difficult to organize child views in a manner that is scalable to various screen sizes without the children overlapping.
- Nevertheless, you may add several children to a FrameLayout and manage their position by providing gravity to each child using the **android:layoutgravity** feature.

```
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <ImageView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        />

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        />

</FrameLayout>
```



TYPES OF UI (USER INTERFACE) LAYOUTS

■ 5. Constraint Layout

- ConstraintLayout permits the creation of complicated layouts with a flat view hierarchy (no nested view groups).
- It is similar to RelativeLayout in that all views are set up according to the relationships between sibling views and the parent layout, but it is more flexible than RelativeLayout and easier to use with the Layout Editor in Android Studio.

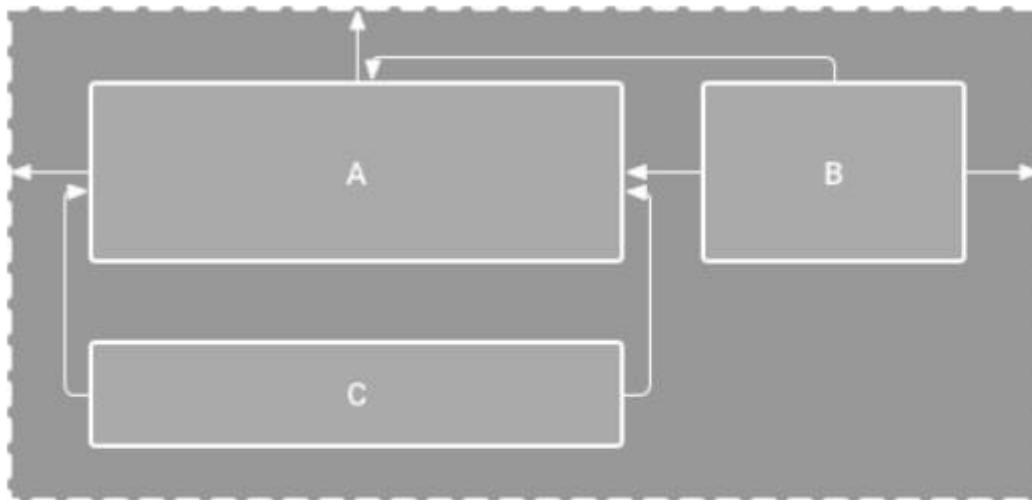


Figure 1. The editor shows view C below A, but it has no vertical constraint

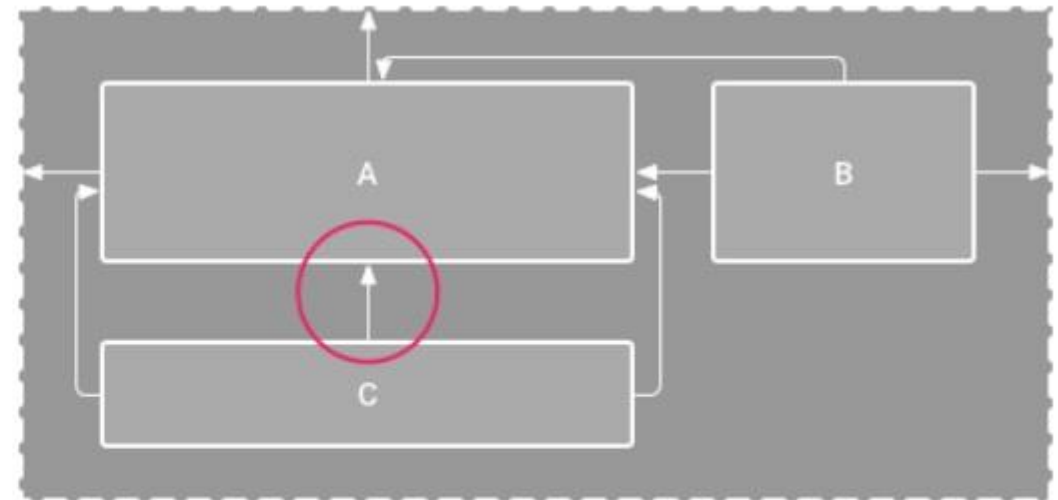
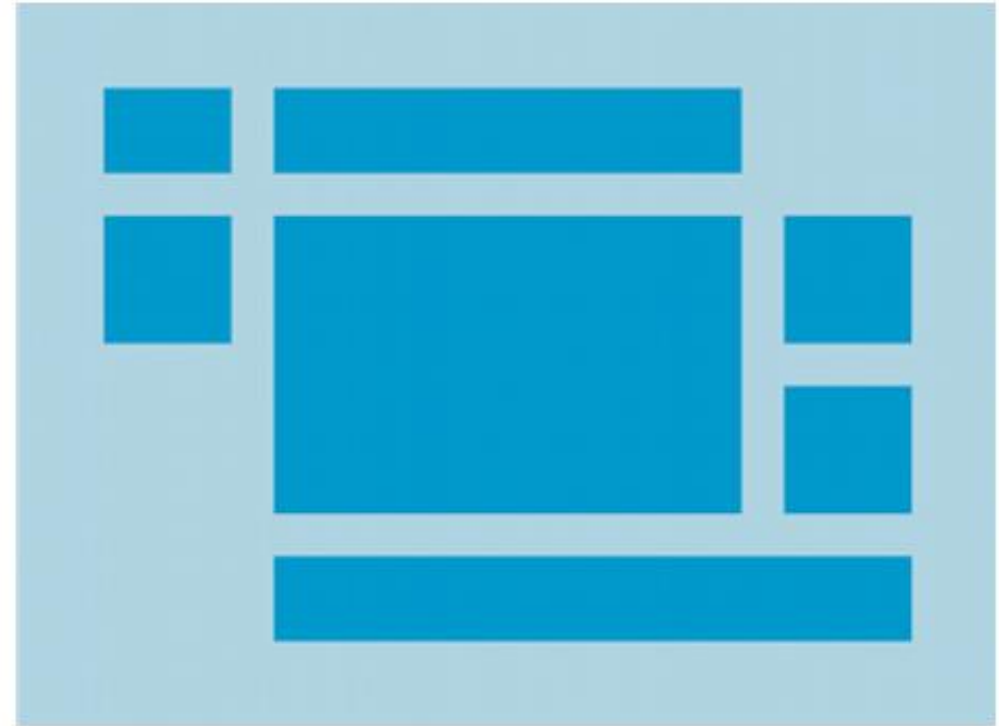


Figure 2. View C is now vertically constrained below view A

TYPES OF UI (USER INTERFACE) LAYOUTS

■ 6. Android Table Layout

- TableLayout is a ViewGroup that presents its children in rows and columns. TableLayout arranges its children into columns and rows.
- TableLayout containers do not display row, column, and cell border lines. The table will have the same number of columns as the row with the most cells. A table can leave cells vacant.



TYPES OF UI (USER INTERFACE) LAYOUTS

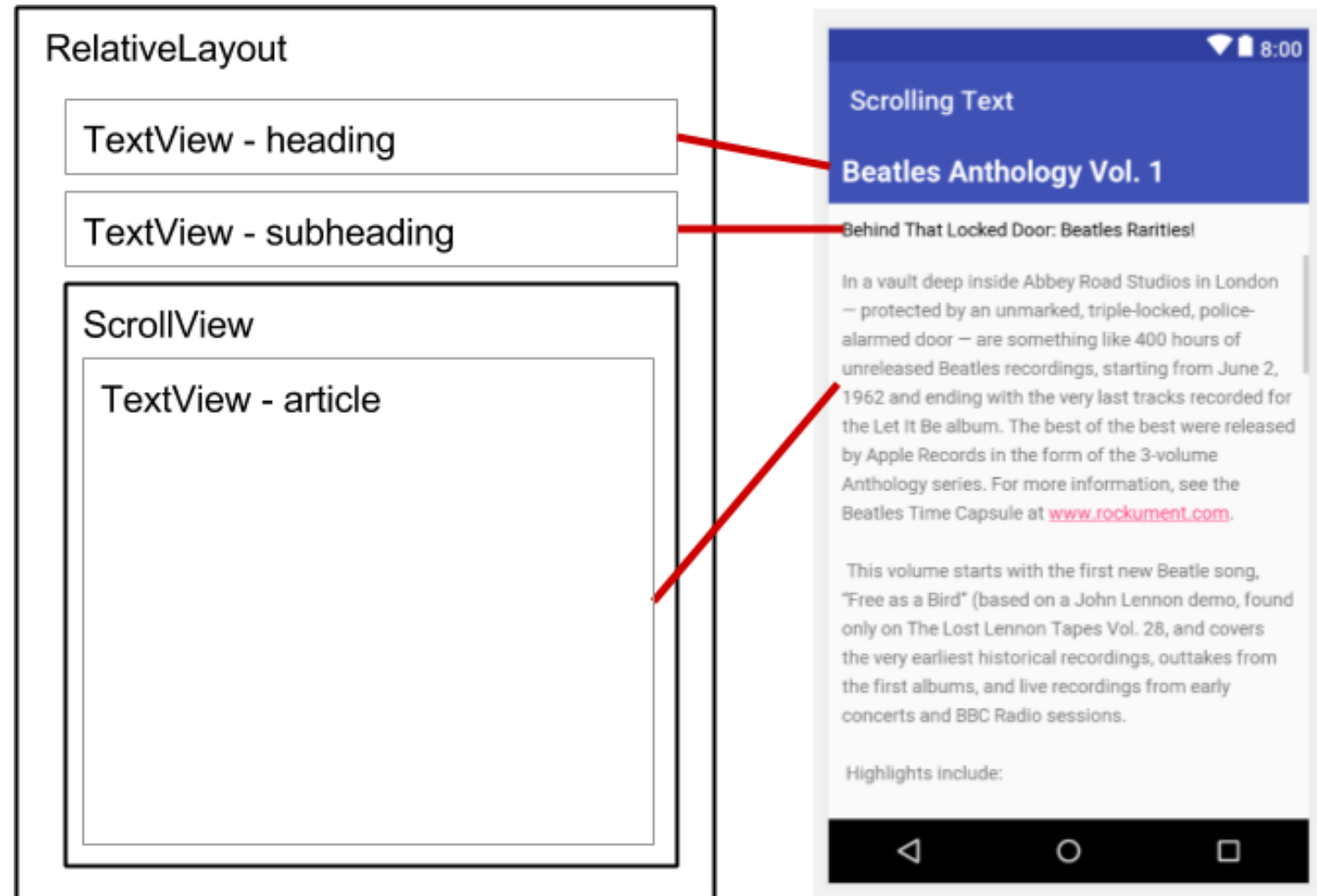
■ 6. Android Table Layout



ROW 1		
ROW 2 COLUMN 1	ROW 2 COLUMN 2	ROW 3 COLUMN 3
ROW 3 COLUMN3	ROW 3 COLUMN 3	

TYPES OF UI (USER INTERFACE) LAYOUTS

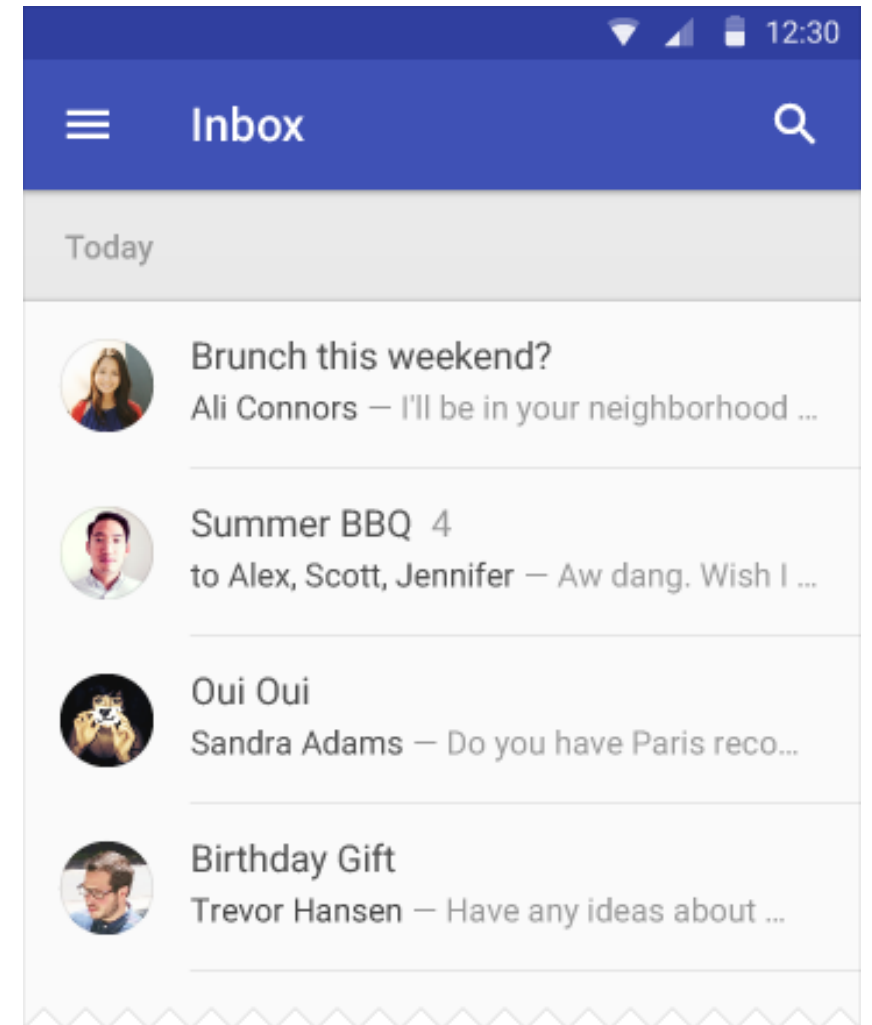
- **7. Scroll View**
- It is a special type of `FrameLayout` that allows users to scroll through a list of views occupying more space than the physical display. The `ScrollView` can contain only one child view or `ViewGroup`, which normally is a `LinearLayout`



TYPES OF UI (USER INTERFACE) LAYOUTS



- **8. List View**
- is a view group that displays a list of scrollable item



TYPES OF UI (USER INTERFACE) LAYOUTS



- **9. Grid View**
- is a View Group that displays items in the two-dimensional scrolling grid.
- The items in the grid come from the ListAdapter associated with this view



COMMONLY USED ANDROID VIEW CLASSES

- TextView
- EditText
- Button
- ImageView
- ImageButton
- CheckBox
- RadioButton
- ListView
- GridView
- DatePicker
- Spinner, etc.

